

PLANEJAMENTO EBT ENTERPRISE

# PLANEJAMENTO DE CÓDIGO EBT PLATFORM

---

Arquitetura, reaproveitamento, módulos, fases, testes, CI/CD e critérios de liberação -  
do Core às verticais

## Documento mestre de referência

Base de planejamento: 06/10/2026

# Como ler este documento

Este é o plano técnico mestre para evitar o erro de programar a plataforma inteira de uma vez. A arquitetura é planejada de ponta a ponta, mas a implementação ocorre em incrementos pequenos. Cada incremento precisa compilar, testar, integrar e demonstrar valor antes da próxima fase.

## Regra de ouro

Planejar tudo -> programar pouco -> testar -> integrar -> homologar -> documentar -> avançar. Nenhuma etapa nova deve começar sobre uma base vermelha.

## 1. Objetivo técnico

- Transformar ativos existentes em uma plataforma EBT reutilizável, sem reescrever tecnologia apenas por preferência de stack.
- Separar Core horizontal de regras específicas de CASST, Vikings e futuras verticais.
- Permitir ativação de módulos por cliente, com isolamento, permissões, auditoria e integrações consistentes.
- Criar um caminho em que novos contratos exijam predominantemente configuração, migração e customização limitada.
- Manter testes automatizados como requisito de continuidade do projeto.

## 2. Baseline atual

| Ativo            | Estado considerado                                                                         | O que pode fornecer ao EBT Core                                                                                              | O que ainda não pode ser assumido                                                        |
|------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| CASST            | Premissa do usuário: quase perfeita                                                        | CRM, contatos, tarefas, atendimento, automações, permissões, integrações, infraestrutura e padrões de UX                     | Necessário revisar acoplamento, licenças, isolamento e generalização antes da extração   |
| Vikings SST main | Código executável até V18, testes automatizados e CI; ainda sem validação operacional real | Pessoa/vínculo/posto, documentos, requisitos, importação, dados sensíveis, treinamentos, EPI, riscos, inspeções, ocorrências | Não tratar como produto homologado; integrações externas e produção permanecem pendentes |
| Vikings V19      | PR draft de fundação de integração RH                                                      | Padrões futuros de reconciliação e sync                                                                                      | Estado atual com testes falhando; não usar como base estável até ficar verde             |

## 3. Decisão de arquitetura

Recomendação: monólito modular em ASP.NET Core/.NET + React/TypeScript, mantendo SQL Server/Azure SQL como linha inicial. Essa decisão reduz retrabalho porque CASST e Vikings já estão próximos dessa família tecnológica. Microserviços devem surgir apenas quando carga, independência de deploy ou risco operacional justificarem a separação.

| Camada     | Tecnologia inicial                                   | Motivo                                                        |
|------------|------------------------------------------------------|---------------------------------------------------------------|
| Frontend   | React + TypeScript                                   | Reaproveitamento e ecossistema já presente                    |
| Backend    | ASP.NET Core / .NET                                  | Base existente em CASST/Vikings e alto desempenho             |
| Banco      | SQL Server / Azure SQL                               | Compatibilidade com ativos existentes e recursos de segurança |
| Arquivos   | Azure Blob ou storage S3 compatível via interface    | Separar binário de metadados e permitir troca de provedor     |
| Cache      | Redis quando houver necessidade medida               | Evitar dependência antes de existir gargalo                   |
| Mensageria | Azure Service Bus/RabbitMQ somente quando necessário | Começar simples; evoluir integrações e jobs com carga real    |

| Camada          | Tecnologia inicial                          | Motivo                                   |
|-----------------|---------------------------------------------|------------------------------------------|
| Identidade      | OIDC/OAuth2/JWT com provedor configurável   | Evitar autenticação proprietária fechada |
| CI/CD           | GitHub Actions + containers                 | Automação já conhecida e auditável       |
| Observabilidade | OpenTelemetry + logs estruturados + alertas | Diagnóstico e SLA desde cedo             |

## 4. Princípios de desenho

1. Módulo possui fronteira clara de domínio, dados, API e permissões.
2. Domínio não depende de controller, UI ou provedor externo.
3. Integrações externas são adapters substituíveis e nunca contaminam a regra central.
4. Toda escrita crítica deve ter idempotência ou mecanismo equivalente quando repetição é possível.
5. Toda decisão sensível deve ser auditável: ator, data, origem, contexto e alteração.
6. Dados clínicos/sensíveis têm políticas específicas e não aparecem por conveniência em dashboards operacionais.
7. Features incompletas ficam atrás de feature flags e não são apresentadas como prontas.
8. Nenhuma regra importante existe apenas no frontend.
9. Migrações de banco são versionadas, revisadas e testadas em ambiente semelhante à produção.
10. Correção de bug começa por teste que reproduz o defeito sempre que tecnicamente viável.

## 5. Arquitetura lógica do EBT Core

| Módulo Core       | Responsabilidade                                               | Origem de reaproveitamento                |
|-------------------|----------------------------------------------------------------|-------------------------------------------|
| Tenancy           | Organização, plano, configuração, isolamento                   | CASST como referência; generalização nova |
| Identity & Access | Usuários, login, papéis, políticas, escopos                    | CASST + Vikings                           |
| People & Context  | Pessoa canônica, vínculos, organizações, relações              | Vikings forte; CASST complementa CRM      |
| Audit             | Trilha de alteração e eventos de segurança                     | CASST + padrões Vikings                   |
| Documents/GED     | Metadados, arquivo privado, hash, versão, categoria, aprovação | CASST + Vikings                           |
| Tasks & SLA       | Responsável, prazo, estado, escalonamento                      | CASST + Vikings                           |
| Protocol          | Numeração e tramitação universal                               | Novo                                      |
| Workflow          | Estados, transições, regras, aprovações, eventos               | Generalização de fluxos CASST/Vikings     |
| Entity Builder    | Entidades dinâmicas e campos configuráveis                     | Novo                                      |
| Form Builder      | Formulários, validação, condição, layout                       | Novo                                      |
| Notifications     | Interno, e-mail, WhatsApp, SMS/push por adapters               | CASST + serviço novo                      |
| Integration Hub   | API keys, OAuth, webhooks, jobs, retries, reconciliação        | CASST + Vikings import/sync               |
| Search            | Busca textual e filtros globais                                | Novo sobre dados do Core                  |
| Analytics         | Datasets, métricas, widgets, dashboards                        | Evolução de dashboards existentes         |

## 6. Estrutura recomendada do repositório

Uma opção de monorepo modular:

```

/src
  /Ebt.Api
  /Ebt.Web
  /Modules
    /Core.Tenancy
    /Core.Identity

```

```

/Core.People
/Core.Audit
/Core.Documents
/Core.Tasks
/Core.Protocol
/Core.Workflow
/Core.Forms
/Core.Notifications
/Core.Integrations
/Core.Search
/Core.Analytics
/Products/Connect
/Products/Flow
/Products/Portal
/Products/Contracts
/Products/Sst
/Products/Legislative
/tests
/Unit
/Integration
/Contract
/E2E
/docs
/architecture
/adr
/runbooks
/acceptance
/infrastructure
/docker
/deployment
/observability

```

## 7. Padrão interno de cada módulo

| Pasta/Camada   | Conteúdo                                               | Regra                                    |
|----------------|--------------------------------------------------------|------------------------------------------|
| Domain         | Entidades, value objects, invariantes, eventos         | Sem dependência de UI/infra externa      |
| Application    | Casos de uso, commands/queries, autorização de negócio | Coordena domínio e interfaces            |
| Infrastructure | EF Core, storage, adapters, fila, API externa          | Substituível por contrato                |
| Api            | Endpoints, DTOs, validação de entrada                  | Sem regra de negócio complexa            |
| Web            | Páginas/componentes do módulo                          | Consome contratos da API                 |
| Tests          | Unitários, integração, contratos e cenários críticos   | Faz parte da entrega, não etapa opcional |

## 8. Multi-tenancy - desenho detalhado

- Todo registro compartilhado entre clientes recebe TenantId obrigatório.
- Tenant é resolvido por identidade autenticada e nunca confiado somente a header enviado pelo cliente.
- Índices únicos que hoje são globais devem, quando apropriado, incluir TenantId.
- Filtros globais de EF Core reduzem risco de consultas sem escopo; dados altamente sensíveis podem receber RLS no SQL Server como defesa adicional.
- Logs e auditoria devem registrar TenantId explicitamente.
- Jobs/filas carregam TenantId no envelope e revalidam contexto ao executar.
- Clientes de alto risco ou exigência contratual podem migrar para database-per-tenant sem alterar contratos de domínio.

## 9. Identidade, RBAC e escopo

| Nível | Exemplo | Implementação |
|-------|---------|---------------|
|-------|---------|---------------|

| Autenticação     | Usuário está autenticado?                                    | OIDC/JWT, expiração, refresh, revogação   |
|------------------|--------------------------------------------------------------|-------------------------------------------|
| Papel            | Admin, RH, Técnico, Auditor                                  | Role simples para agrupamento             |
| Permissão        | Documents.Review, Contract.Approve                           | Policy explícita por ação                 |
| Escopo           | Somente unidade/posto/cliente X                              | Filtro servidor + autorização por recurso |
| Regra de negócio | Aprovador não pode aprovar próprio evento em casos definidos | Validação no caso de uso/domínio          |

## 10. Banco de dados e padrões de persistência

- Usar GUID/UUID para identidade global quando compatível com a base; chaves externas ficam em tabela própria e nunca substituem ID interno.
- Campos CreatedAt, UpdatedAt e versão/concurrency token em agregados críticos.
- Soft delete somente quando houver razão de negócio; histórico regulatório deve preferir estados e vigências.
- Eventos temporais usam início/fim e nunca sobrescrevem passado quando mudança de contexto é relevante.
- Arquivos não ficam em varbinary no banco salvo exceção justificada; banco armazena metadados, hash, storage key, tamanho, MIME e classificação.
- Transações delimitam alterações que precisam ser atômicas.
- Outbox pattern deve ser introduzido quando eventos externos/filas passarem a exigir garantia entre commit e publicação.

## 11. API - convenções

| Tema          | Padrão                                                                   |
|---------------|--------------------------------------------------------------------------|
| URLs          | /api/v1/{module}/{resource}; evitar verbos quando REST for suficiente    |
| Erros         | ProblemDetails padronizado com code, message, traceId e detalhes seguros |
| Paginação     | cursor ou page/size definido por recurso; limites máximos                |
| Idempotência  | Idempotency-Key em comandos repetíveis/rede instável                     |
| Concorrência  | ETag/version ou token equivalente em updates críticos                    |
| Auditoria     | CorrelationId/TraceId e actor em operações relevantes                    |
| Versionamento | Contrato explícito; breaking change exige nova versão ou migração        |
| Webhooks      | assinatura, timestamp, retry, deduplicação e DLQ/reconciliação           |

## 12. GED - desenho de referência

11. Upload cria registro em estado pendente e valida extensão/MIME/tamanho.
12. Arquivo recebe hash e é armazenado em storage privado.
13. Antivírus/scan é executado antes de disponibilização ampla quando aplicável.
14. Metadados indicam tenant, categoria, entidade relacionada, versão, autor e classificação de acesso.
15. Aprovação/rejeição gera evento e nunca altera silenciosamente o arquivo anterior.
16. Download revalida permissão por recurso; URL temporária não substitui autorização.
17. Retenção e eliminação são políticas versionadas e auditadas.
18. Assinatura eletrônica/digital é adapter separado; resultado e evidência de assinatura ficam persistidos.

## 13. Workflow Engine - evolução em seis versões

| Versão | Entrega                                   | Critério de aceite                                                    |
|--------|-------------------------------------------|-----------------------------------------------------------------------|
| W1     | Estado inicial, etapas e transição manual | Fluxo simples persiste histórico e bloqueia transição inválida        |
| W2     | Papéis de etapa e responsáveis            | Somente ator autorizado executa cada transição                        |
| W3     | Condições e branching                     | Mesma entrada produz caminho determinístico conforme regra versionada |
| W4     | Aprovações, rejeição e retorno            | Histórico completo e regra de segregação quando configurada           |
| W5     | Timers, SLA, eventos e notificações       | Atraso e escalonamento são testáveis sem depender de relógio real     |
| W6     | Designer e versionamento                  | Nova versão não muda processos já iniciados sem migração explícita    |

## 14. Protocolo universal

- Sequência única por tenant e exercício/tipo conforme configuração.
- Geração transacional para impedir número duplicado sob concorrência.
- Interessado, assunto, origem, sigilo, anexos e unidade responsável.
- Tramitação com recebimento, envio, despacho e histórico.
- Integração opcional com Workflow sem transformar protocolo em engine de workflow.
- Consulta externa somente para protocolos marcados como públicos e com token/identificação adequada.

## 15. Entity Builder e Form Builder

| Componente        | Capacidade v1                                | Capacidade madura                                    |
|-------------------|----------------------------------------------|------------------------------------------------------|
| Entity Builder    | Nome, campos básicos, tipos, obrigatoriedade | Relacionamentos, índices, regras e templates         |
| Form Builder      | Campo, seção, ordem, validação simples       | Condição, repetição, permissões e layouts            |
| Record Engine     | CRUD e pesquisa de registros dinâmicos       | Eventos, workflow, documentos e auditoria integrados |
| Schema Versioning | Versão do modelo                             | Migração assistida entre versões                     |

Recomendação: não usar Entity Builder para domínios altamente regulados como prontuário ou contabilidade sem desenho específico. Dinamismo excessivo pode trocar velocidade inicial por complexidade de manutenção.

## 16. Notifications e Integration Hub

| Área             | Requisitos obrigatórios                                                                      |
|------------------|----------------------------------------------------------------------------------------------|
| Notificações     | template versionado, destinatário, canal, estado, tentativas, erro, opt-out quando aplicável |
| E-mail           | provider adapter, bounce/erro, correlação                                                    |
| WhatsApp         | adapter oficial, templates, janela/categoria, custos, webhooks, idempotência                 |
| Webhooks saída   | assinatura, retry exponencial, timeout, DLQ                                                  |
| Webhooks entrada | assinatura, replay protection, deduplicação                                                  |

| Área          | Requisitos obrigatórios                                               |
|---------------|-----------------------------------------------------------------------|
| APIs externas | OAuth/API key, secret store, rate limit, retry e circuit breaker      |
| Importação    | preview, validação, commit explícito, reconciliação e reprocessamento |
| Sincronização | external identity, conflitos revisáveis e histórico                   |

## 17. Observabilidade e segurança desde a primeira release

| Domínio      | Implementar desde o início                                              |
|--------------|-------------------------------------------------------------------------|
| Logs         | estruturados, sem dados sensíveis desnecessários, traceId/tenant/module |
| Métricas     | latência, erro, throughput, fila, jobs, storage, DB                     |
| Tracing      | OpenTelemetry para requests e integrações                               |
| Alertas      | falha de health, erro repetido, backup, fila parada, SLA crítico        |
| Segredos     | Key Vault/secret manager; nunca repositório/appsettings commitado       |
| Upload       | limites, MIME, scan, nome seguro, storage privado                       |
| Web          | CSP, headers seguros, proteção XSS/CSRF conforme arquitetura            |
| Dependências | scanner e atualização planejada                                         |
| Backups      | backup automático + restore testado                                     |
| Auditoria    | eventos administrativos, acesso sensível e alterações críticas          |

## 18. Estratégia de testes

| Camada             | Objetivo                                 | Exemplo                                           |
|--------------------|------------------------------------------|---------------------------------------------------|
| Unitário           | Regra pura e rápida                      | mudança de estado, cálculo de SLA, aplicabilidade |
| Application        | Caso de uso com dependências controladas | aprovar documento, abrir protocolo                |
| Integração DB      | EF + SQL real/testcontainer              | índices, concorrência, transações, filtros tenant |
| Contrato API       | Formato e códigos HTTP                   | cliente não quebra por mudança involuntária       |
| Integração externa | Sandbox/fake protocolar                  | Meta, e-mail, storage, eSocial                    |
| E2E                | Fluxo completo no navegador              | login -> criar -> aprovar -> consultar            |
| Segurança          | Acesso proibido e enumeração             | tenant B não lê tenant A; ID direto é negado      |
| Carga              | Capacidade e regressão                   | listagens, upload, dashboard, filas               |
| Restore            | Recuperação real                         | banco + arquivos + chaves/config                  |

## 19. Pipeline CI/CD obrigatório

Fluxo mínimo recomendado:

```
push / pull request
-> format/lint
-> typecheck
-> build
```

- > unit tests
- > integration tests
- > security/dependency scan
- > frontend build
- > container build
- > deploy staging
- > E2E / smoke
- > aprovação de release
- > produção
- > health / smoke pós-deploy
- > rollback automático/manual se necessário

### Gate

Main deve permanecer verde. PR com teste novo falhando é evidência de trabalho em andamento, não funcionalidade pronta.

## 20. Estratégia Git e branches

- main: sempre potencialmente liberável.
- feature/<produto>-<meta>: escopo pequeno e rastreável.
- PR obrigatório para mudanças de domínio, banco, segurança e integração.
- Commits preferencialmente descrevem teste/regra/implementação; evitar mega-commit.
- ADR obrigatório para decisões que alteram arquitetura, tenancy, autenticação, persistência ou contratos públicos.
- Tag/release identifica exatamente o código implantado em cada cliente/ambiente.

## 21. Definição de pronto (Definition of Done)

19. Critérios de aceite escritos e aprovados.
20. Código compilando sem erro; warnings novos avaliados.
21. Testes novos e regressão passando.
22. Autorização servidor validada, inclusive acesso por ID direto.
23. Migração de banco criada e testada quando aplicável.
24. Logs/telemetria suficientes para diagnosticar falhas.
25. Documentação de operação e configuração atualizada.
26. Feature flag/configuração definida quando liberação é gradual.
27. Smoke/E2E executado em staging.
28. Sem segredo, dado real indevido ou dependência local acidental no repositório.
29. Rollback ou estratégia de correção conhecida.

## 22. Plano de implementação - Core

| ID | Fase                           | Dependência             | Saída principal                                                     | Horas  |
|----|--------------------------------|-------------------------|---------------------------------------------------------------------|--------|
| C0 | Congelar baseline e inventário | CASST/Vikings revisados | Snapshot, dependências, direitos de uso, mapa de módulos            | 20-35h |
| C1 | Fundação EBT                   | C0                      | Solution/monorepo, CI, Docker, config, health, observabilidade base | 25-40h |
| C2 | Tenancy                        | C1                      | Tenant, resolução, filtros, índices, testes de isolamento           | 35-55h |
| C3 | Identity/RBAC                  | C1-C2                   | OIDC/JWT, papéis, permissões, escopo por recurso                    | 30-45h |
| C4 | Audit                          | C2-C3                   | Evento de auditoria, actor, before/after seguro, consultas          | 20-35h |
| C5 | People & Context               | C2-C4                   | Pessoa, vínculo, organização, relações reutilizáveis                | 25-45h |

| ID  | Fase                   | Dependência | Saída principal                                            | Horas   |
|-----|------------------------|-------------|------------------------------------------------------------|---------|
| C6  | Documents/GED v1       | C3-C5       | upload privado, hash, categoria, aprovação, versão, acesso | 50-75h  |
| C7  | Tasks/SLA              | C3-C5       | responsável, prazo, estado, escalonamento base             | 30-50h  |
| C8  | Protocol               | C3-C7       | numeração concorrente, tramitação e histórico              | 35-55h  |
| C9  | Workflow W1-W4         | C3-C8       | estados, transições, regras, aprovação                     | 70-110h |
| C10 | Notifications          | C7-C9       | in-app + e-mail; adapters e retries                        | 30-50h  |
| C11 | Integration Hub v1     | C5-C10      | webhooks, API keys/OAuth, jobs, reconciliação              | 45-75h  |
| C12 | Entity/Form Builder v1 | C4-C11      | modelos dinâmicos, formulários, validações, registros      | 80-120h |
| C13 | Search/Analytics v1    | C5-C12      | busca, datasets, métricas e widgets básicos                | 40-65h  |

As horas são esforço econômico aproximado para planejamento e podem cair com reaproveitamento validado. Não devem ser somadas cegamente sem revisão do código real de CASST e Vikings.

## 23. Marcos vendáveis do Core

| Marco | Componentes mínimos              | Produto que já pode ser vendido               |
|-------|----------------------------------|-----------------------------------------------|
| M1    | C0-C5 + tarefas básicas          | Connect/CRM e aplicações internas simples     |
| M2    | + Documents + Tasks + Protocol   | EBT Flow inicial                              |
| M3    | + Workflow W1-W4 + Notifications | Flow profissional e Contracts básico          |
| M4    | + Integration Hub                | Implantações com legado e canais externos     |
| M5    | + Builders + Analytics           | Plataforma configurável para múltiplos nichos |

## 24. Plano de produto - EBT Connect

| Incremento | Entrega                                     | Teste de aceite                            | Horas marginais      |
|------------|---------------------------------------------|--------------------------------------------|----------------------|
| CON1       | Contatos, organizações, histórico e tarefas | Criar, relacionar, filtrar e auditar       | 25-40h               |
| CON2       | Inbox multiusuário                          | Fila, atribuição, histórico e concorrência | 30-50h               |
| CON3       | WhatsApp oficial adapter                    | Webhook, template, status, retry, custo    | 35-60h + homologação |
| CON4       | Automação/chatbot                           | Regra, trigger, ação, fallback humano      | 40-70h               |
| CON5       | Relatórios e campanhas                      | Permissão, métricas e opt-out aplicável    | 25-45h               |

## 25. Plano de produto - EBT Flow

| Incremento | Entrega                          | Dependência         | Horas marginais |
|------------|----------------------------------|---------------------|-----------------|
| FL1        | Protocolo + GED + tarefas        | Core M2             | 30-45h          |
| FL2        | Workflow configurável básico     | C9                  | 35-55h          |
| FL3        | Formulários/entidades dinâmicas  | C12                 | 30-50h          |
| FL4        | SLA/escalonamento/assinatura     | C7/C10 + adapter    | 35-60h          |
| FL5        | Portal externo de acompanhamento | Identity + Protocol | 25-45h          |
| FL6        | Dashboard operacional            | C13                 | 20-35h          |

## 26. Plano de produto - EBT Portal

| Incremento | Entrega                        | Crítérios principais                     | Horas                    |
|------------|--------------------------------|------------------------------------------|--------------------------|
| PO1        | CMS páginas/notícias/menu      | SEO, preview, versionamento              | 35-55h                   |
| PO2        | Documentos/publicações         | busca, categorias, acessibilidade        | 25-40h                   |
| PO3        | Transparência/e-SIC/ouvidoria  | protocolos, status, privacidade          | 40-70h                   |
| PO4        | Dados abertos                  | datasets, CSV/JSON, metadata             | 30-50h                   |
| PO5        | Acessibilidade/analytics/cache | eMAG/WCAG conforme contrato, performance | 25-45h                   |
| PO6        | Migração e importadores        | reconciliação e logs                     | 20-40h por origem típica |

## 27. Plano de produto - EBT Contracts

| Fase | Escopo                                                         | Quando vender                        |
|------|----------------------------------------------------------------|--------------------------------------|
| CT1  | Fornecedor, contrato, vigência, responsável, documento, alerta | Assim que Core M3 estiver estável    |
| CT2  | Fiscalização, medição, aditivo, garantia, ocorrências          | Após primeiro cliente piloto         |
| CT3  | DFD/ETP/TR e workflow de contratação                           | Após regras públicas revisadas       |
| CT4  | Licitação/proposta/PNCP adapters                               | Somente com integração homologada    |
| CT5  | IA para rascunho/análise                                       | Depois de dados e governança maduros |

## 28. Plano de produto - EBT SST

Se Vikings for operacionalmente validado, EBT SST deve nascer por extração e generalização, não por reescrita.

| Passo | Ação                                                 | Gate                                                |
|-------|------------------------------------------------------|-----------------------------------------------------|
| SST1  | Homologar fluxos reais no Vikings                    | Usuários reais executam cenários e aprovam          |
| SST2  | Fechar integrações externas prioritárias             | RH/clínica/SSO/storage homologados                  |
| SST3  | Separar configuração Vikings do domínio reutilizável | Segundo tenant/cliente sintético roda sem hard-code |

| Passo | Ação                                    | Gate                                                     |
|-------|-----------------------------------------|----------------------------------------------------------|
| SST4  | Migrar capacidades comuns para EBT Core | Dois consumidores automatizados evitam regressão         |
| SST5  | Criar pacote EBT SST                    | Onboarding, configuração, licença, suporte, documentação |

## 29. Plano de produto - EBT Legislativo

| Fase | Escopo                                                        | Observação                                     |
|------|---------------------------------------------------------------|------------------------------------------------|
| LEG1 | Vereadores, legislaturas, comissões, proposições e documentos | Aproveita People/GED/Workflow                  |
| LEG2 | Tramitação, pareceres, pautas e agendas                       | Aproveita Flow                                 |
| LEG3 | Sessões, presença, ordem do dia e atas                        | Domínio específico                             |
| LEG4 | Votação eletrônica e painel                                   | Exige disponibilidade e testes de concorrência |
| LEG5 | Portal legislativo/Diário/assinaturas                         | Integra Portal e assinatura                    |
| LEG6 | Mobile e integrações                                          | Somente após web madura                        |

## 30. Educação e Saúde - regra de entrada

### Não antecipar

Educação e Saúde não devem ser construídas apenas para perseguir um único edital. Entrar quando houver demanda repetida, cliente âncora, especialista de domínio e capacidade de manutenção regulatória.

| Vertical | Primeiro pacote vendável                           | Só depois                                                      |
|----------|----------------------------------------------------|----------------------------------------------------------------|
| Educação | alunos, turmas, professores, matrícula, frequência | notas/diário, Censo, integrações oficiais, app                 |
| Saúde    | cadastro, agenda, fila, atendimento administrativo | prontuário, farmácia, regulação, faturamento, e-SUS/vigilância |

## 31. Backlog técnico priorizado

| Prioridade | Item                            | Por quê                                 |
|------------|---------------------------------|-----------------------------------------|
| P0         | Baseline/reuse audit            | Bloqueia arquitetura e evita duplicação |
| P0         | Tenancy + Identity + Audit      | Base de segurança para todos os módulos |
| P0         | Documents + Tasks               | Valor horizontal imediato               |
| P0         | Protocol + Workflow v1          | Base de Flow e governo                  |
| P1         | Notifications + Integration Hub | Necessário para operação real           |
| P1         | Connect productization          | Receita rápida com CASST                |
| P1         | Portal productization           | Editais e privados                      |
| P1         | Vikings operational validation  | Desbloqueia SST/reuso forte             |

| Prioridade | Item                | Por quê                                         |
|------------|---------------------|-------------------------------------------------|
| P2         | Entity/Form Builder | Aumenta configurabilidade                       |
| P2         | Contracts           | Vertical adjacente de alto reaproveitamento     |
| P2         | Legislativo         | Somente quando POC relevante puder ser atendida |
| P3         | AI Service          | Depois de dados/processos confiáveis            |
| P3         | Education/Health    | Cliente âncora e domínio especializado          |

## 32. Matriz de validação por módulo

| Módulo         | Código verde    | Homologação interna | Cliente piloto    | Produção estável    | Pode virar Core           |
|----------------|-----------------|---------------------|-------------------|---------------------|---------------------------|
| CASST base     | Premissa        | Quase               | Sim/uso existente | Premissa do usuário | Após auditoria de reuso   |
| Vikings V1-V18 | Sim no main     | Parcial             | Não confirmado    | Não                 | Só após piloto + extração |
| Vikings V19    | Não no PR atual | Não                 | Não               | Não                 | Não                       |
| EBT Core novo  | A construir     | A construir         | A construir       | A construir         | Após dois consumidores    |

## 33. Checklist de homologação de cada incremento

30. Happy path completo.
31. Erro de validação e dados ausentes.
32. Usuário sem permissão.
33. Usuário de outro tenant/escopo.
34. Repetição/idempotência quando aplicável.
35. Concorrência quando duas pessoas atuam no mesmo registro.
36. Persistência após reload/restart.
37. Auditoria e logs.
38. Falha de integração externa.
39. Backup/restore para dados críticos.
40. Responsividade e acessibilidade aplicável.
41. Métricas/alerta mínimo para produção.

## 34. Padrão de correção de bugs

Todo bug relevante deve seguir o ciclo: reproduzir -> escrever teste que falha -> corrigir a menor causa possível -> executar regressão -> revisar impacto em migração/API -> documentar quando afetar operação. Evitar "corrigir na tela" um erro cuja causa está na regra ou no banco.

## 35. IA/Codex como multiplicador, não como substituto de gates

- Tarefas para IA devem ser fechadas e pequenas: um módulo, uma regra, um conjunto de testes.
- Prompt deve declarar explicitamente o que NÃO deve ser alterado.
- IA não pode considerar tarefa concluída apenas porque compilou; precisa de testes e critérios de aceite.
- Mudança grande deve ser dividida em PRs independentes quando possível.
- Review humano/arquitetural é obrigatório para segurança, tenancy, dados sensíveis, migrações e contratos externos.
- Logs de CI e testes são fonte de verdade superior a uma afirmação textual de "funcionando".

## 36. Estimativa global de esforço

| Bloco                               | Com CASST reutilizável | Se Vikings também for validado | Comentário                                                                               |
|-------------------------------------|------------------------|--------------------------------|------------------------------------------------------------------------------------------|
| EBT Core reutilizável               | ~300-500h              | ~220-380h                      | Diferença vem da generalização de documentos, processos, vínculo, importação e segurança |
| Primeira versão comercial Core/Flow | ~150-250h adicionais   | ~100-180h adicionais           | Sem builders maduros e verticais completas                                               |
| Connect                             | ~80-140h               | ~70-120h                       | Maior reaproveitamento CASST                                                             |
| Portal                              | ~120-200h              | ~110-180h                      | Maioria do esforço é CMS/transparência/migração                                          |
| Contracts                           | ~180-300h              | ~150-260h                      | Aproveita Flow/GED                                                                       |
| Legislativo                         | ~250-400h              | ~220-350h                      | Regras específicas e POC                                                                 |
| Educação                            | ~400-700h              | ~380-650h                      | Vertical própria                                                                         |
| Saúde municipal                     | ~700-1.300h+           | ~650-1.200h+                   | Muito além de saúde ocupacional SST                                                      |

## 37. Ordem recomendada de execução

42. Auditar e congelar CASST/Vikings como fontes de reuso.
43. Criar fundação EBT e Tenancy/Identity/Audit.
44. Extrair People, Documents e Tasks em contratos reutilizáveis.
45. Construir Protocol e Workflow v1.
46. Colocar EBT Flow inicial em staging e homologar.
47. Productizar Connect em paralelo por cima do Core.
48. Productizar Portal e usar como porta de entrada pública.
49. Criar Notifications e Integration Hub.
50. Validar Vikings operacionalmente e transformar partes comprovadas em EBT SST/Core.
51. Criar Entity/Form Builder somente depois que fluxos fixos comprovarem os padrões.
52. Construir Contracts.
53. Construir Legislativo quando houver capacidade de POC.
54. Adicionar IA como camada assistiva.
55. Entrar em Educação/Saúde apenas com tese comercial e domínio suficientes.

## 38. Linha do tempo de produto

| Janela       | Resultado esperado                                  | Venda possível                                |
|--------------|-----------------------------------------------------|-----------------------------------------------|
| 0-4 semanas  | baseline + fundação + tenancy/auth/audit            | manter vendas CASST/serviços existentes       |
| 5-8 semanas  | documents/tasks/protocol básicos                    | Flow piloto + Connect consolidado             |
| 9-12 semanas | workflow v1 + notifications + staging sólido        | Flow vendável + Portal inicial                |
| 3-6 meses    | Portal, integrations, analytics e reuso consolidado | Connect/Portal/Flow com implantação repetível |
| 6-12 meses   | Contracts + SST productizado se validado            | contratos médios e verticais adjacentes       |
| 12-24 meses  | Legislativo e builders maduros                      | POCs maiores e plataforma mais configurável   |

A linha do tempo é uma referência de sequencing, não promessa de prazo. O fator dominante será o estado real dos repositórios, disponibilidade de homologação e quantidade de pessoas trabalhando em paralelo.

## 39. Critérios para não avançar

- Main/CI vermelho.
- Bug de isolamento entre tenants ou escopos.
- Migração de banco não testada.
- Backup sem restore comprovado.
- Integração marcada como pronta sem homologação externa.
- Feature crítica com apenas teste unitário e nenhum teste de integração quando depende de banco/rede.
- Dado sensível acessível a perfil indevido.
- Módulo novo exigindo cópia de regras existentes em vez de extração/generalização.
- Prazo comercial obrigando pular gate de segurança/qualidade.

## 40. Critérios para extração de um componente para o Core

- 56. Existe em pelo menos dois produtos/consumidores reais ou um segundo consumidor sintético robusto.
- 57. A API do componente não contém vocabulário específico de um cliente.
- 58. Configuração substitui hard-code de marca, tenant e domínio.
- 59. Testes demonstram comportamento igual nos consumidores.
- 60. Migração e compatibilidade foram planejadas.
- 61. Ownership e versão do componente estão definidos.
- 62. A extração reduz duplicação real; não é abstração antecipada sem uso.

## 41. Operação em produção

| Área        | Runbook mínimo                                     |
|-------------|----------------------------------------------------|
| Deploy      | quem aprova, como implanta, smoke, rollback        |
| Banco       | backup, restore, migrations, rollback/forward fix  |
| Storage     | retenção, recuperação, antivírus/scan              |
| Identidade  | revogação, incidente de acesso, rotação de segredo |
| Integrações | fila, retry, reconciliação, credencial expirada    |
| SLA         | classificação P1/P2/P3, comunicação, escalonamento |
| Incidente   | registro, impacto, causa, correção, post-mortem    |
| Privacidade | solicitação de titular, incidente, retenção        |

## 42. Encerramento técnico

### Arquitetura final desejada

A EBT não deve possuir sete sistemas completamente independentes. Deve possuir um Core estável e versionado, com produtos que reutilizam identidade, tenancy, pessoas, documentos, tarefas, workflow, notificações, integrações, auditoria e analytics. O código específico existe apenas onde o domínio realmente exige.

A decisão mais importante do plano é preservar o que já foi aprendido em CASST e Vikings. Reescrever tudo aumentaria risco e prazo; extrair, testar e generalizar cria um ativo acumulativo. O desenvolvimento deve permanecer incremental e guiado por critérios de aceite, exatamente como o Vikings já começou a fazer em vários PRs.